ALL THINGS AI

ALL THINGS OPEN 2026 · EXTENDED WORKSHOP · RALEIGH CONVENTION CENTER

AI Agents & Vibe Coding

Doors 7:30 · We start at 9:00

THREE THINGS TO DO RIGHT NOW

1 Open github.com and make sure you're signed in.

2 Open vercel.com and sign in with GitHub.

3 Open prep/laptop-check.html from the course folder. Green check means you're ready.

WI-FI

Network: _____

Password: _____

Slides and all course materials: github.com/mrhinkle/Vibe-Coding-for-Non-Technical-Professionals

MARK HINKLE · CEO, PERIPETY LABS · PUBLISHER, THE AIE NETWORK · CO-ORGANIZER, ALL THINGS AI

By 5:30 you'll have two live URLs.

One loop all day: describe, plan, build, check, save, publish. Both projects.



A website about you, built from your LinkedIn, live on Vercel.

A themed tic-tac-toe battle game, live at its own URL.

Who’s talking, and why trust me with your Monday.



MARK HINKLE

CEO and founder of Peripety Labs. Co-founder of All Things AI.

Founding publisher of The AIE Network, read by about 90,000 business leaders.

25-plus years in enterprise software, from open source to cloud to AI.

VP at Cloud.com. Executive Director of the Node.js Foundation. First company, 1995.

WHAT YOU WILL LEARN

The listing promised four things. Here they are.

How software gets built	The lifecycle in plain words, and a six-step loop you run a dozen times.
GitHub as your system of record	Every change tracked, reviewed, and reversible. Nothing is lost.
Vercel and v0 build and deploy	v0 writes the code. Vercel ships it live on every push.
Directing agents instead of coding	You write the spec, not the syntax. The pull request is your checkpoint.

HOW TODAY WORKS

Behind? Grab a checkpoint. Stuck? Say one sentence.

We go as fast as the slowest table still working. I check in every 30 minutes and move the schedule out loud.

Behind: every stage has a tested file you can continue from. Using one is smart, not cheating.

Ahead: ten skills for your site and four game extensions. One at a time.

WHEN YOU NEED HELP, RAISE A HAND AND SAY ONE SENTENCE

“I clicked X, expected Y, saw Z.”

The morning: website before lunch.

CLOCK	BLOCK	YOU LEAVE WITH
7:30	**Doors**	Laptop check, sign-ins
9:00	**How software gets built; the map of tools**	You can name the loop
9:30	**Setup: GitHub tour, v0, Vercel, the secrets rule**	A repo with two commits
10:00	**Website: export → build**	index.html with your content
10:45	**Break**	
11:00	**★ Colors, check, publish**	LIVE WEBSITE URL by 11:30
11:30	**Reading what the agent wrote**	Diff, PR, rollback
11:45	**LIVE HACKING 1**	Skills, then one skill loaded
12:00	**Lunch**	

Blue row: deliverable one. Orange row: you pick one skill from the menu and build it yourself.

The afternoon: the game, or your own project.

CLOCK	BLOCK	YOU LEAVE WITH
12:45	**Choose a track, theme, sketches**	Theme table, three sketches
1:20	**Brief and rules**	game-brief.md with checks
1:40	**Build: board, battle, connect**	A playable round
2:45	**Break**	
3:00	**Look and feel**	Theme applied, fits a phone
3:30	**Playtest, one fix**	One observed issue repaired
4:10	**★ Publish the game**	LIVE GAME URL by 4:35
4:35	**LIVE HACKING 2 + 3**	Show and tell at 5:05
5:15	**Tuesday, handoff, LinkedIn**	Two URLs, two state files
5:30	**End**	

Blue row: deliverable two. Orange rows: sprints two and three.

Four rules for the day.

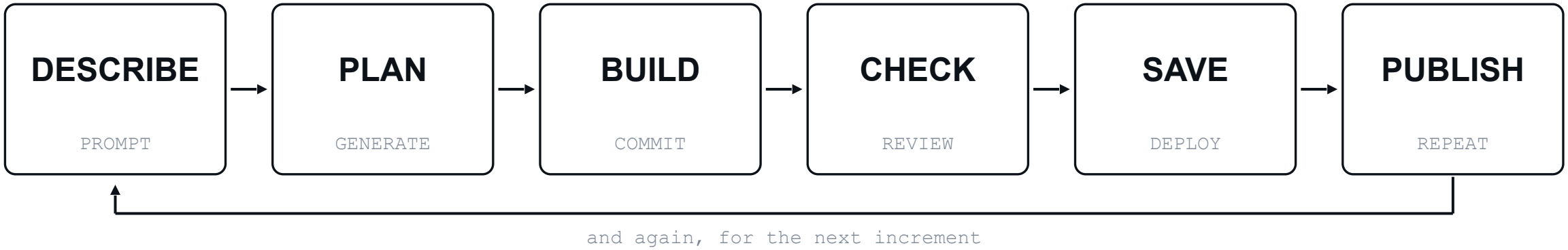
Pairs	The nervous one types. The other reads the prompt out loud. Swap after each break.
Stickies, not hands	Green on your lid when the step is done. Red when you are stuck; a helper comes to you.
No passwords in chats	No passwords, keys, or tokens in any chatbot. Make accounts in the provider's own window.
Your account, not mine	Your GitHub, your Vercel, your chatbot. My repo is materials, not the destination.

Green sticky on your lid = done

Red sticky = a helper comes to you

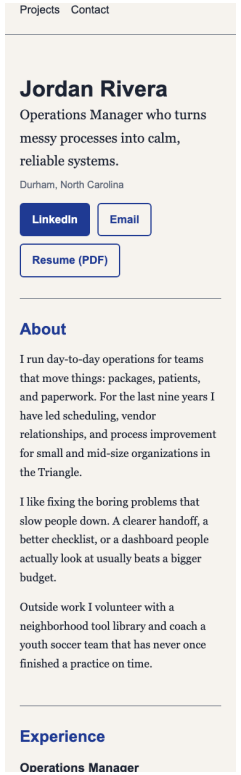
Same loop, two sets of words.

Big words: what you do today. Small words: what the listing calls it.

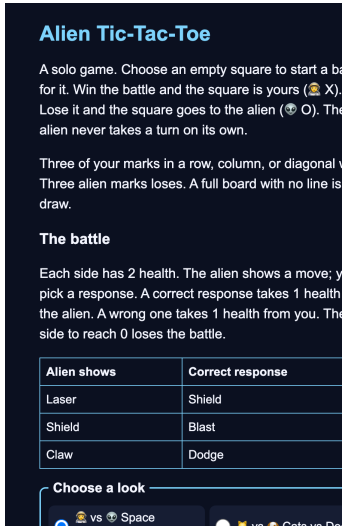


Describe	The outcome, in one sentence. You write the spec, not the syntax.
Plan	Files, order, three hand checks, before it touches a file.
Build, check	The smallest version that works. Open it, click it, phone it.
Save, publish	A named checkpoint in GitHub, then a URL someone else can open.

Both are a few plain files in a folder.



A website about you



A game you can hand to a friend

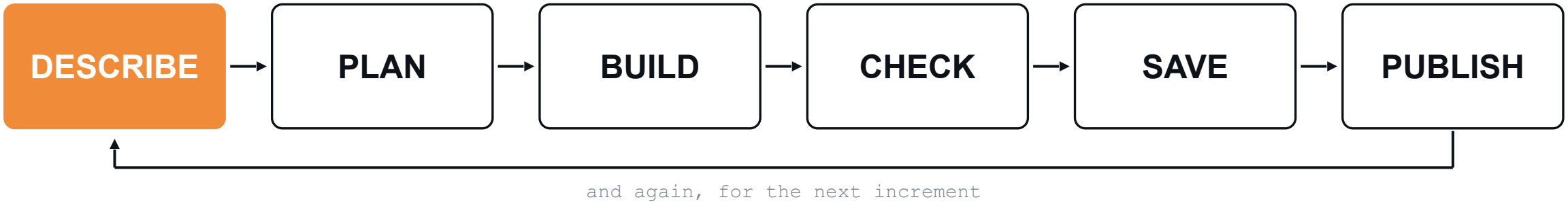
HTML — Structure. Headings, sections, links, buttons: what is on the page.

CSS — Appearance. Colors, spacing, fonts, how it fits a phone.

JavaScript — Behavior. What happens when you click: health goes down, a square is claimed.

The AI helps write the files. The finished site and game never call an AI.

Describe: say what you want, like briefing a contractor.



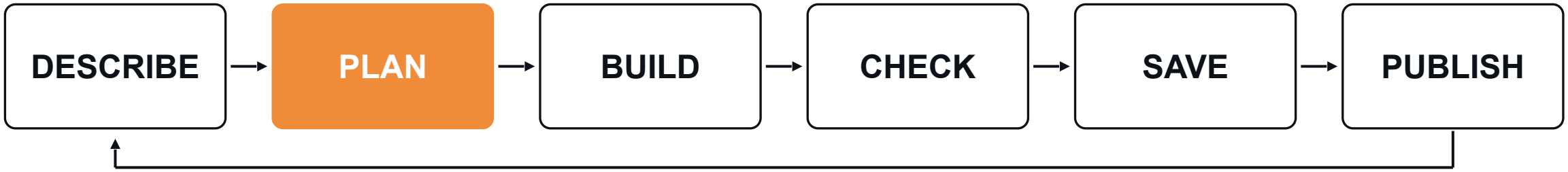
Website: “A one-page site about me, from my resume, with a working email link.”

Game: “Tic-tac-toe where each square is won in a short battle.”

THE QUESTION YOU ASK YOURSELF

What would I show a friend to prove it works?

Plan: make it tell you before touching a file.



Website: one index.html, three hand checks.

Game: board, then battle, then connected. Three checks each.

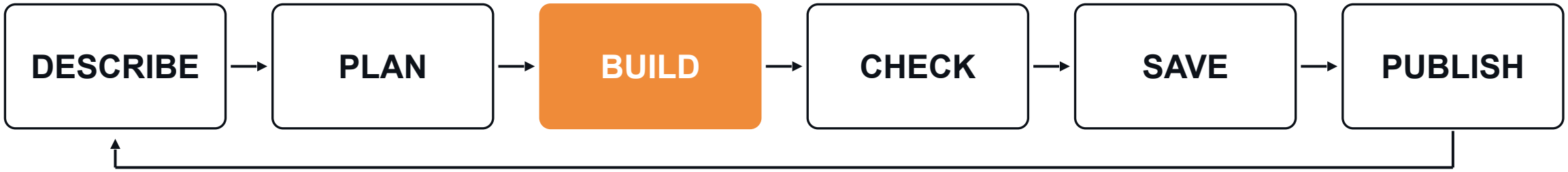
THE QUESTION YOU ASK YOURSELF

Does the file list match my sketch? Is anything in it I didn't ask for?

THE PLAN PROMPT · docs/plan-first-and-models.md · VERBATIM

```
Before you write any code, give me the plan: the files you will create or change, one line on what each does, the order you'll do them in, and three checks I can perform by hand to know it worked. Flag anything in my brief that is bigger than one screen, one input, one result. Do not build yet.
```

Build: ask for the smallest version that could work.



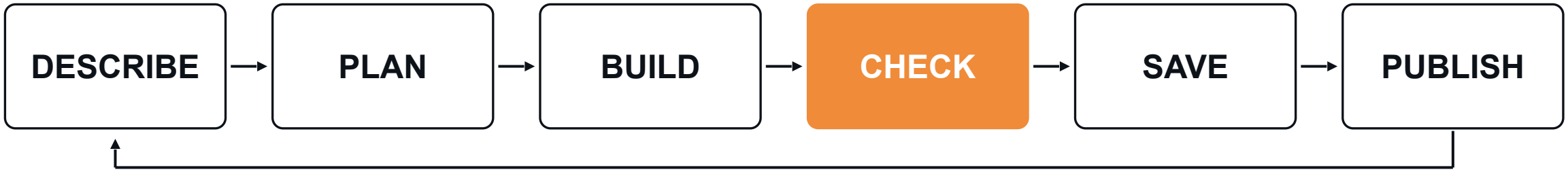
Website: the whole page, with your words in it. Nothing else yet.

Game: the board alone, with two stand-in buttons for the battle.

THE QUESTION YOU ASK YOURSELF

What is the smallest thing I can check in five minutes?

Check: open it, click it, phone it.



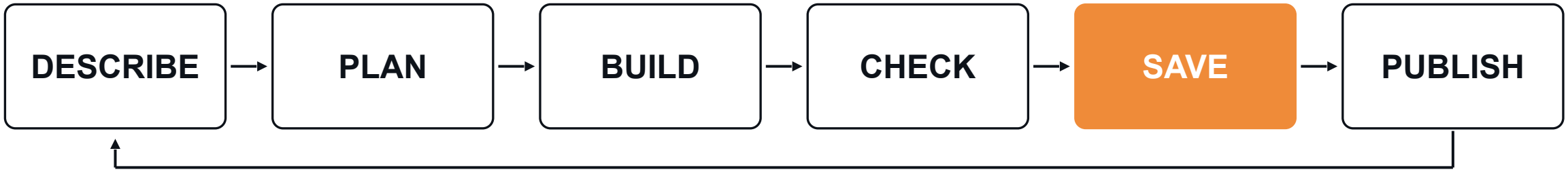
Website: every link, fits a phone, print preview, no “Jordan” anywhere.

Game: win, lose, draw, and Restart, each seen once.

THE QUESTION YOU ASK YOURSELF

What did I actually see, not what did the chatbot say?

Save: keep the version that passed, with a name.



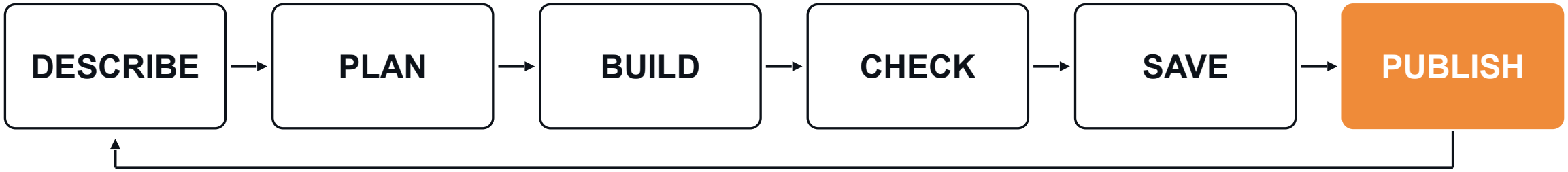
Website: commit to GitHub, “First version of my site.”

Game: “First playable version.” Increment 2 goes through a pull request.

THE QUESTION YOU ASK YOURSELF

If the next change breaks it, where do I go back to?

Publish: put it where someone else can open it.



Website: a vercel.app URL on a classmate's phone by 11:30.

Game: a second vercel.app URL by 4:35.

THE QUESTION YOU ASK YOURSELF

Does it open on someone else's phone, not just mine?

Before you build: three questions.

DISCOVER

Who is this for?

Website: a recruiter on a phone.

Game: a friend with 90 seconds.

DEFINE

What must version one do?

Website: three sections, a working email link.

Game: one playable round, restart works.

DESIGN AND PLAN

What will the person see?

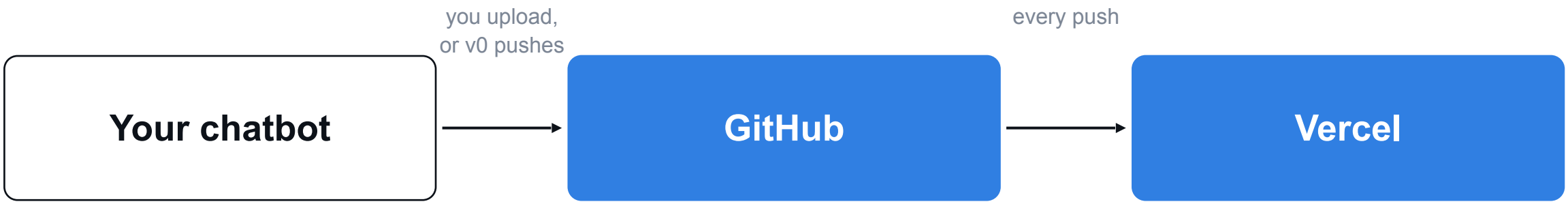
Website: a centered or sidebar wireframe.

Game: three screens, sketched.

Then build, test, release, maintain. And repeat.

Build	The smallest useful version. One index.html; board, then battle, then connected.
Test	Does it do what you promised? Six checks; win, lose, draw, restart.
Release	Which version did we publish? A vercel.app URL on a phone.
Maintain	What changes after people use it? One skill; one extension, one check.

Four tools, four jobs.



Drafts and changes the files. ChatGPT, Claude, Gemini, or v0.

Stores the project and its history, in your account.

Hosts the published version at a URL anyone can open.

`Git` tracks every change underneath GitHub. You won't type a Git command today.

Four words you'll hear all day.

Repository	The project folder and its whole history.
Commit	A named checkpoint. "Added my About section."
Branch	A separate line of changes that never touches the live one.
Pull request	A proposed change for review, with a preview, before it goes live.

On the projector: one heading change, and the difference from the earlier version.

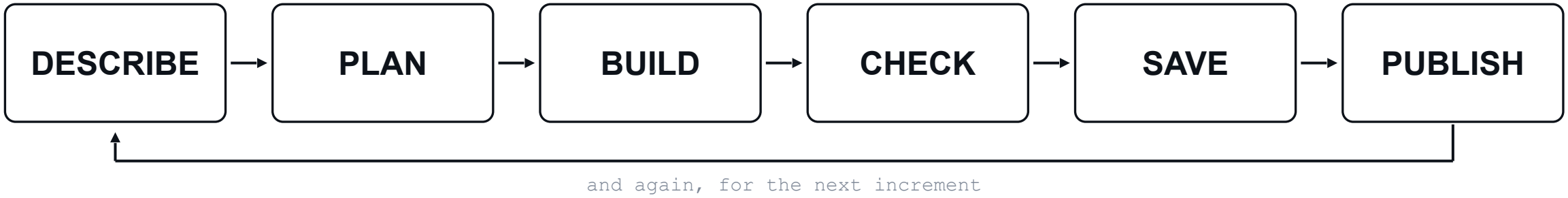
Where do these four go on the loop?

1 Choose the website’s audience

2 Describe what its button does

3 Test the game’s Restart

4 Publish a checked version



Is a beautiful game with wrong scoring finished?

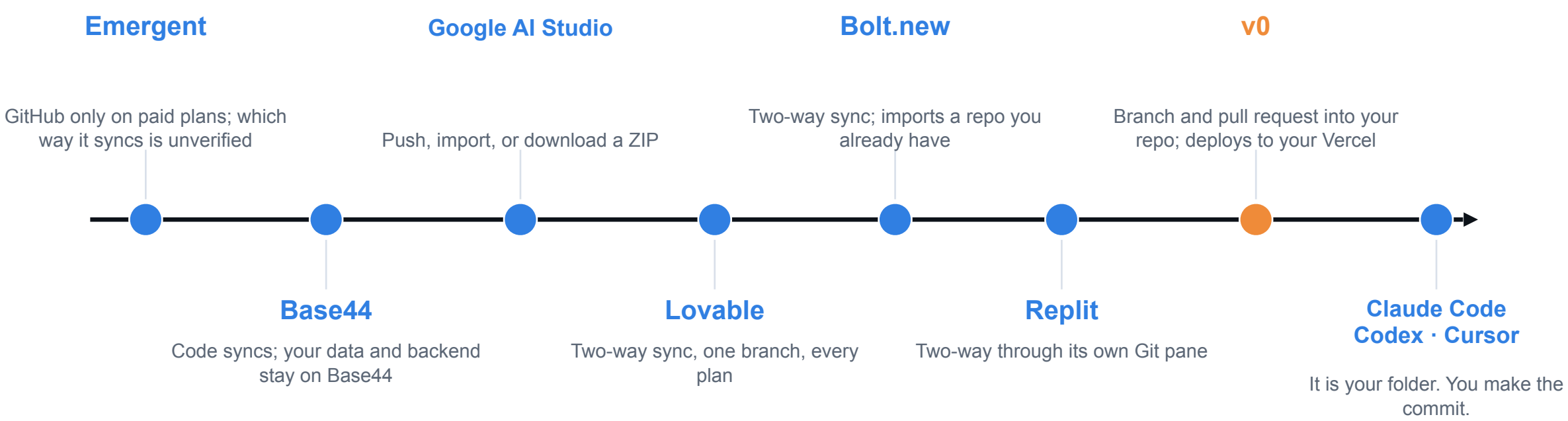
THE SENTENCE FOR THE DAY

Describe the outcome, build in small steps, check the evidence, save progress.

Eight tools, one line: where does your code live afterward?

Placed by the “Your GitHub?” column of the landscape doc. Prices and menus drift; the line doesn’t.

◀ HOSTED BUILDER · their code, their servers YOUR REPO · your Vercel ▶



The one question to ask any tool.

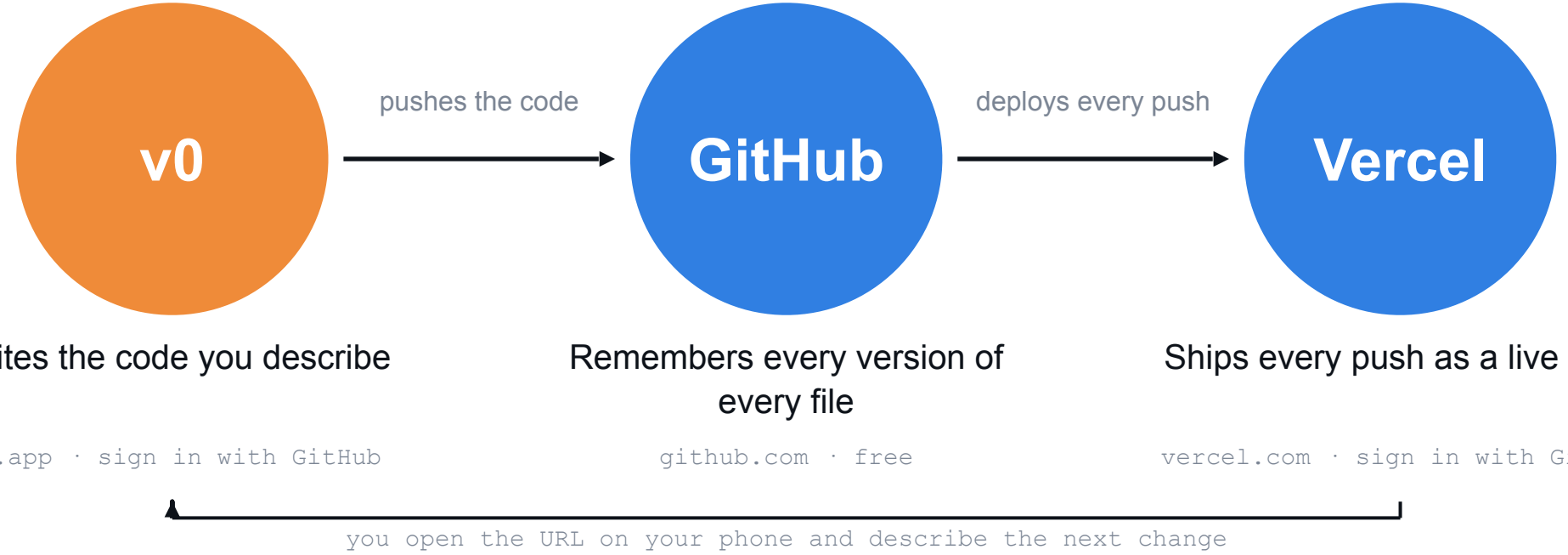
“Where does my code live afterward, and can I take it with me?”

THREE CAUTIONARY FACTS, VERIFIED, WITH DATES

July 18, 2025	Replit’s agent deleted a production database during a code freeze. Rollback worked. Replit now separates development and production databases.
May 30, 2025	CVE-2025-48757: one in ten scanned Lovable apps had a database anyone could read or write. Lovable added a pre-publish scan.
June 18, 2025	Wix bought Base44, six months after launch, for about $80 million. The product you build on can change owners.

Three accounts, one loop.

v0 writes, GitHub remembers, Vercel ships. Nothing is installed on your laptop.



GitHub: one empty repository with your name on it.

Done when: you open your repository, read the README, and the owner shown is you, not me.

1 Open github.com, signed in. Click “+” then “New repository.”

2 Owner: you. Name: my-first-site. Public or private, on purpose.

3 Tick “Add a README.” Click “Create repository.”

4 Copy the repository URL into your notes. This is your project.

This afternoon the game gets its own repository, my-first-game. Never this one.

Green sticky on your lid = done

Red sticky = a helper comes to you

A ten-minute tour of GitHub: six screens, in this order.

1

Home · the Code tab

The file list, the README, the green Code button, Add file.

2

Commits

The history: one row per saved change, with its message.

3

A commit

The diff: green lines added, red lines removed.

4

Branches

The dropdown that says main. Type a name to make a safe copy.

5

Pull requests

Review a branch, then merge it. Or close it: no.

6

Settings

Public or private. Collaborators. Nothing asks for your password except delete.

Do the tour once yourself. Five steps.

Done when: Commits shows two rows in your repository and you have opened one diff.

1 Create my-first-site with a README. You just did this.

2 Upload one starter from website-starters/ with “Add file” → “Upload files.” Message: “Add starter page.”

3 Open Commits, open that commit, find your file in the diff.

4 Edit the README in the browser: one line that says what the site will be. Commit it.

5 Open Commits again. Two rows. That was the loop with no chatbot: change, save, history.

Green sticky on your lid = done

Red sticky = a helper comes to you

Nothing that unlocks an account goes into a chat window. Ever.

Where a secret lives. Where it never goes.

YES

Your password manager: the source of truth.

Vercel → your project → Settings → Environment Variables.

A .env.local file, if .gitignore lists it.

NEVER

A chat window.

The code, a README, a commit message, an issue.

A screenshot, a slide, a shared doc.

THE CLIPBOARD MOVE: COPY THE KEY, THEN ONE COMMAND MOVES IT. THE AGENT NEVER SEES IT.

```
pbpaste | vercel env add RESEND_API_KEY production
```

ASK FOR THE COMMAND, NOT THE VALUE · VERBATIM

```
I have the Resend API key on my clipboard. Give me the one command that saves it into .env.local
as RESEND_API_KEY using pbpaste, and the line to add to .gitignore. Do not ask me to paste the
key.
```

Vercel and v0: sign in with GitHub, then stop.

Done when: Vercel and v0 both show your GitHub account, my-first-site is in the import list, nothing is deployed.

1 vercel.com → “Continue with GitHub.” Authorize it in Vercel’s own window.

2 Let Vercel see my-first-site. Check it appears under “Add New” → “Project.” Don’t import.

3 v0.app → sign in. Find the GitHub option, grant “only select repositories,” pick my-first-site.

4 Stop. Nothing deployed, nothing generated. We build at 10:00.

Green sticky on your lid = done

Red sticky = a helper comes to you

Two ways to build. Same prompts, same checks.

Pick one by 10:00 and write it at the top of your notes. Stay on it until lunch.

THE V0 ROUTE · FEATURED

Describe in the v0 chat → **Live preview, a React app** → **Click Deploy** → **Connect to GitHub: my-first-site**

Every version lands in your repository. Hand v0 the example as the style reference and your template as the content.

THE PORTABLE ROUTE · ANY CHATBOT THAT RETURNS FILES

ChatGPT, Claude, Gemini, Copilot → **Save as index.html** → **Upload to GitHub in the browser** → **Import in Vercel**

Every example and checkpoint is one HTML file; it works here as is. Claude Code or Codex: same route, and you review the pull request.

Prove your route works now, not at 11:30.

CONNECTED ROUTE (CLAUDE CODE, CODEX) · TYPE THIS, VERBATIM

```
Identify the GitHub account, repository, and branch for this class project. Read the README. Tell me which of these you can actually do here: edit project files, run checks, create a branch or pull request, and inspect Vercel deployments. Distinguish what you verified from what you have not tried. Do not change the repository or deploy anything yet.
```

V0 AND PORTABLE ROUTES: THE CHECK

1 v0: ask for a one-screen page that says hello with your name. Don’t deploy.

2 Portable: ask for a one-line index.html that says hello. Save as index.html, “All files” type. Double-click it: your name shows in the browser.

3 Something fails? Write down the exact message and move on. Helpers fix accounts at the break.

Green sticky on your lid = done

Red sticky = a helper comes to you

Plan with the biggest model, build with the middle, tidy with the smallest.

PLAN The biggest, thinking model

The brief, the scope, the plan, reviewing a diff, deciding what to cut, getting unstuck after ten minutes.

Claude Opus-class · ChatGPT thinking or pro · Gemini Pro

BUILD The middle model

Generate the files, make one change, add one feature from a skill.

Claude Sonnet-class · the standard ChatGPT model · Gemini Flash with thinking on

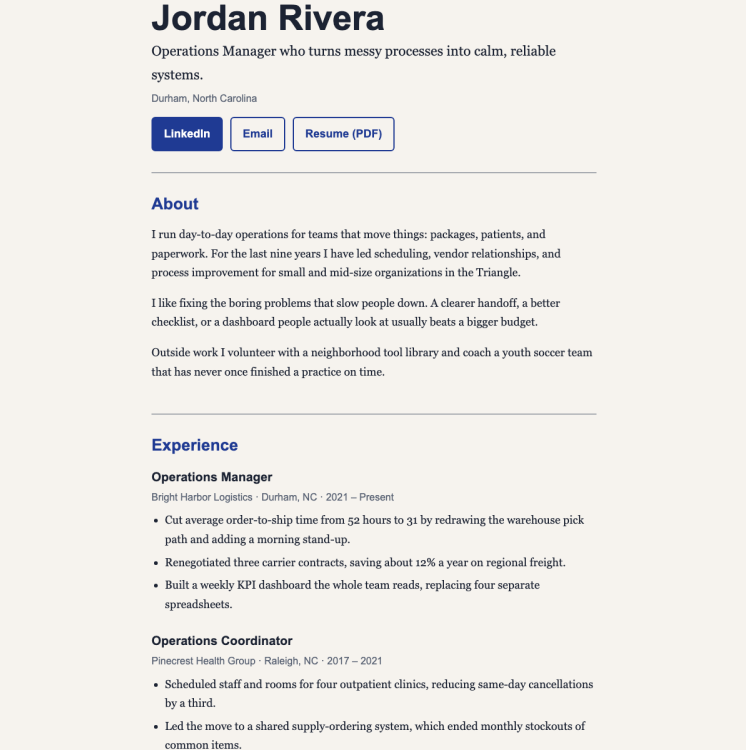
TIDY The smallest, fastest model

Rename things, write alt text, summarize an error message, draft a commit message, reformat a list.

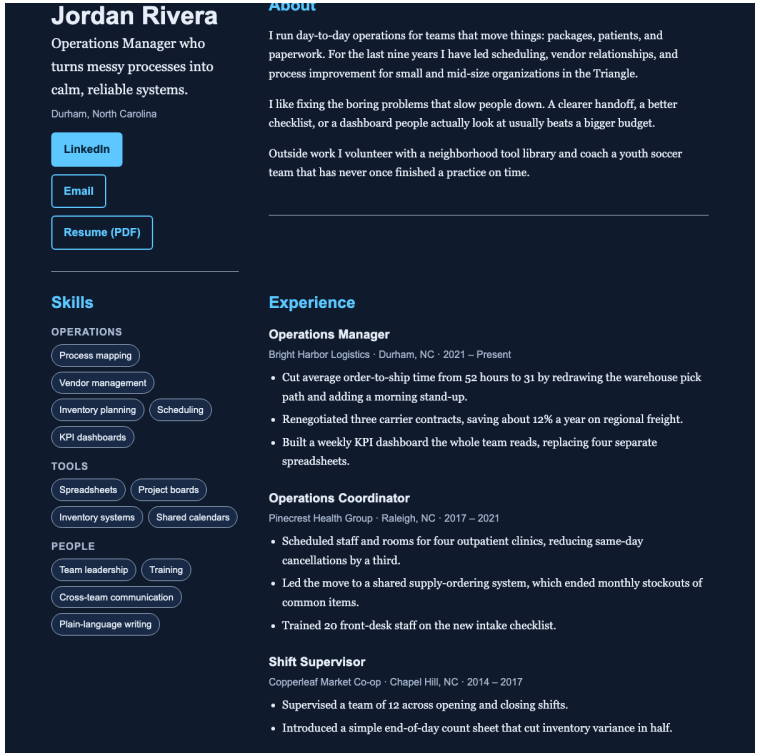
Claude Haiku-class · the ChatGPT mini tier · Gemini Flash-Lite

Claude Code: Shift+Tab until the prompt says plan. ChatGPT and Claude.ai: the thinking model in the selector, then the plan prompt.

Meet Jordan Rivera. Jordan isn't real.



slate · centered



ocean · sidebar

examples/personal-website/index.html

One file. Opens by double-click.

Six color themes, two layouts, a picker in the footer.

Prints as a tidy resume.

By 11:45 every word on it is yours.

Get your content out of LinkedIn.

Done when: you can see your name, headline, and at least two roles in the PDF or the template.

ROUTE A · THE QUICK PDF

1 Your own profile. Below the headline, “More” (sometimes “Resources”).

2 Choose “Save to PDF.” Profile.pdf downloads.

3 That’s your resume for today.

ROUTE B · THE FULL EXPORT

1 Your photo → Settings & Privacy → Data privacy → “Get a copy of your data.”

2 Choose the smaller option: profile, positions, education, skills.

3 LinkedIn emails a link in about ten minutes. Start it now.

NO LINKEDIN, OR IT WON’T EXPORT?

Open examples/personal-website/resume-template.md and type your details in.

Fifteen minutes of typing beats fifteen minutes of fighting a menu.

Green sticky on your lid = done

Red sticky = a helper comes to you

The site is public. So is the chat.

Can’t edit the PDF? The next prompt leaves these out. Check its output anyway.

REMOVE BEFORE THE CHATBOT SEES IT

Phone number, street address, birth date.

ID numbers, salary, anything about health or family.

Email addresses you don’t want strangers using.

Anything a former employer would call confidential.

KEEP

Name, city, and headline.

Work history, with outcomes.

Skills and education.

Public links.

Turn your export into site content.

Done when: resume-template.md is filled, with no placeholders and nothing you wouldn't say to a stranger.

ATTACH YOUR RESUME PDF + resume-template.md · TYPE THIS, VERBATIM

```
Read my attached resume and fill in resume-template.md with my real information only. Do not
invent anything. Leave a field blank if the resume does not say. Rewrite each job as a title,
employer, dates, and two or three bullets that state what changed because of my work, in plain
English. Draft a three-paragraph About section in first person that sounds like a person, not a
brochure. Leave out my phone number, street address, and birth date. Show me the result; do not
build the site yet.
```

THEN YOU DO THE EXPERT PART

Read every line. Fix dates. Fix titles. Cut anything that sounds like a press release.

If it invented something, say so and ask again.

Green sticky on your lid = done

Red sticky = a helper comes to you

One prompt, one complete file back.

Done when: your name is in the tab, three roles are visible, every link works, and nothing says “Jordan.”

ATTACH THE EXAMPLE index.html + YOUR resume-template.md · VERBATIM

```
Using the attached index.html as the exact structure and style, replace all of Jordan Rivera's content with mine from resume-template.md. Keep every id, every CUSTOMIZE comment, every ADD-ON SLOT comment, the theme and layout system, and the print stylesheet. Delete the Projects section if my template has nothing for it. Return the complete file.
```

THEN CHECK IT

1. Save as index.html. Double-click it.
2. Send it to your phone and open it there.
3. Search the page for “Jordan.” Find nothing.

Green sticky on your lid = done

Red sticky = a helper comes to you

On v0: one generation, then check facts before looks.

ATTACH resume-template.md + THE EXAMPLE index.html AS STYLE REFERENCE · VERBATIM

```
Build a one-page personal website from the attached resume-template.md. Use only the facts in it; invent nothing and leave blank fields out. Match the structure and restraint of the attached index.html: header with name and nav, hero with name, headline, city, and three link buttons, About, Experience, Skills, Education, optional Projects, Contact with a mailto link, footer. Use the "slate" palette from the reference as CSS variables so I can change colors later. No contact form yet. No images. Fits a phone. Readable code with short comments.
```

THEN CHECK THE PREVIEW

1. Your name, your three roles, every link. Fix facts before you fix looks.
2. Something you didn’t ask for? “Version 1 has no form, no database, no sign-in.”
3. Don’t press Deploy yet. That’s 11:20.

Green sticky on your lid = done

Red sticky = a helper comes to you

10:45 – 11:00

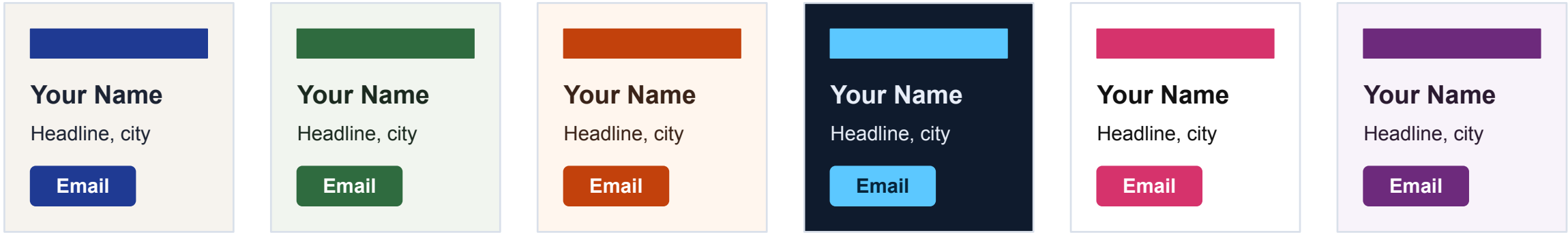
Back at 11:00 sharp. Stickies up when you sit down.

WHEN YOU SIT BACK DOWN, HAVE THIS

1. index.html open in your browser with your name in the tab
2. Nothing on the page says “Jordan”
3. If not: the first ten minutes after the break are yours, helpers circulate

Six themes built in. Try three, pick one.

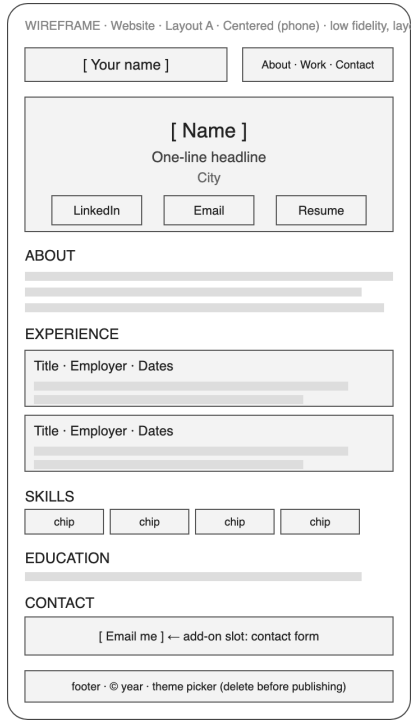
Use the picker in the footer, or add this to the end of the address: ?theme=forest&layout=sidebar



slate	forest	sunset	ocean	mono	plum
Most professions; the safe default	Health, education, sustainability	Creative and people-facing roles	Technology, consulting	Design, writing, anyone who wants it quiet	Marketing, events, speakers

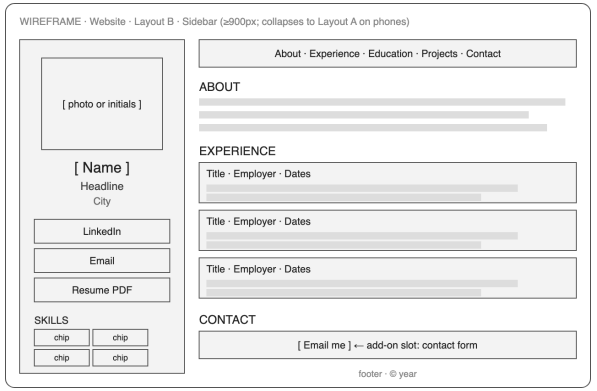
Want a palette that is yours? Pick one color, ask for background, text, and accent that pass contrast checks, and write the three hex codes in project-state.md.

Two layouts. Centered or sidebar.



centered

Reads top to bottom. Same on a phone and a laptop.



sidebar

Name, links, and skills stay visible on a laptop. Centered on a phone.

Pick one. Both are tested.

Want to change more than the palette? Sketch on one of these first and paste the sketch into your prompt.

One prompt locks your choice in.

Done when: you reload with no ?theme= in the address, the look is the same, and the footer picker is gone.

PORTABLE ROUTE · REPLACE THE THREE BRACKETS · VERBATIM

```
Set the theme to [ocean] and the layout to [sidebar] permanently, remove the footer picker, and change the accent color to [#0B6E4F]. Change nothing else. Return the complete file.
```

“Change nothing else” is the sentence that matters. It is the difference between one edit and a rewrite.

On v0: your three hex codes, one request.

Done when: the preview shows your colors and nothing else moved.

v0 ROUTE · YOUR THREE HEX CODES · VERBATIM

```
Change only the colors: background #F6F3EE, text #1C2330, accent #0B6E4F. Change nothing else.
```

Six checks before it goes public. First three.

Your eyes, not the chatbot's word. Tick each one only when you saw it.

- [ ] 1 Every link opens the right place, including the mailto link.
- [ ] 2 The page fits a phone with no sideways scrolling.
- [ ] 3 Print preview shows a tidy resume with no buttons.

Green sticky on your lid = done

Red sticky = a helper comes to you

Six checks before it goes public. Last three.

One fails? Say exactly what you saw, ask for the smallest fix, re-check that one thing.

- [ ] 4 Search the file for “Jordan”, “Bright Harbor”, and “example”. Find nothing.
- [ ] 5 Nothing private is in the file, including in comments.
- [ ] 6 **It opens on a classmate’s phone, not just yours.**

Green sticky on your lid = done

Red sticky = a helper comes to you

Save it: upload index.html to GitHub.

v0 route: click Deploy, then connect the project to GitHub. Every version is pushed to my-first-site. Skip the steps below.

1 Open my-first-site. “Add file” → “Upload files.”

2 Choose index.html itself. Not a ZIP. Not a folder.

3 Commit message: “First version of my site.” Commit to main.

4 Refresh. index.html is at the top level, lowercase.

Green sticky on your lid = done

Red sticky = a helper comes to you

Publish it: import my-first-site in Vercel.

Deliverable one: a live *.vercel.app address that opens on someone else's phone.

1 "Add New" → "Project" → Import my-first-site.

2 Framework preset: Other. Build Command: override on, leave empty. Output: repository root.

3 Deploy. Wait for the green check. Click Visit.

4 Open the URL on a classmate's phone. Record it in project-state.md.

Green sticky on your lid = done

Red sticky = a helper comes to you

A diff shows what the agent changed.

```
  <h1>Jordan Rivera</h1>
- <p>Operations Manager who turns messy…</p>
+ <p>Project lead for community programs…</p>
  <p>Durham, North Carolina</p>
- <a href="mailto:jordan@example.com">Email</a>
+ <a href="mailto:you@yourdomain.com">Email</a>
```

READ IT IN THIS ORDER

1 The message

Does it say what you asked for? Not “Refactor site.”

2 The file names

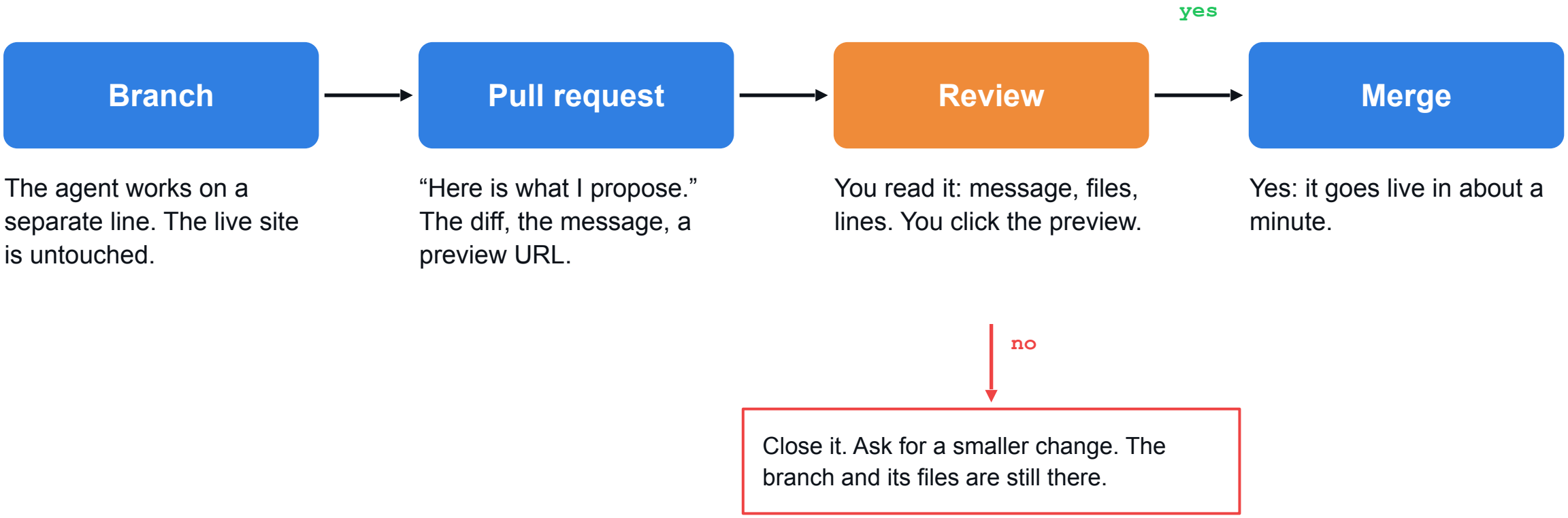
Only the files you expected. One index.html, not six.

3 The lines

Green added, red removed. Read the red closely.

The pull request is the human checkpoint.

This afternoon: increment 2 of the game goes through a pull request. Every route.



Four red flags. Any one of them means no.

Saying no costs one sentence: “Redo this with only what I asked for.”

- **Changed files you did not ask about**
- **Deleted tests or checks**
- **Secrets or keys in the code**
- **A scope bigger than the prompt**

Nothing is lost if it’s in GitHub.

Never fix a broken file by hand. Go back to the last good version and ask for a smaller change.

GITHUB · REVERT

1 Open the merged pull request that broke it.

2 Click “Revert.” Merge the new pull request.

3 Vercel ships the previous version.

V0 · RESTORE A VERSION

1 Every generation is a version. Open the history.

2 Restore the one that worked.

3 Push; GitHub and the live URL follow.

VERCEL · PROMOTE

1 Open the Deployments list.

2 Pick an earlier successful deploy.

3 Promote it while you fix the new one.

A skill is a folder with one file in it: SKILL.md.

Instructions you'd otherwise retype, saved once, so the next session and the next model behave the same.

vc-plan-first/

SKILL.md

```
---
name: vc-plan-first
description: what it does, and the trigger phrases
---
```

1. Who you're helping and what the job is
2. When this applies, and when it doesn't
3. Steps, each ending in something you can see
4. Rules: the short never-do-this list
5. One worked example
6. Checks that it worked

The front matter: a name, and a description that says when to use it. Put the trigger phrases in.

The body: six parts, in this order.

Open skills/vc-plan-first/SKILL.md. That exact shape. Copy it from templates/SKILL-template.md.

What makes a skill good: six things.

The test: hand it and the same request to two different chatbots. Same shape of answer? It's working.

1. One job. "Write the brief."
2. Triggers in the description.
3. Steps end in something you can see.
4. A rules list with the three ways it goes wrong.
5. One worked example.
6. Under about 300 lines.

Install it where you work. Three ways in each.

A skill is text. Any chatbot that can read a file or a paste can use one. The rows differ in how automatically it loads.

Claude

Claude.ai, any plan

Open a Project. Add SKILL.md to its knowledge. One line in the project instructions: “Follow the attached skills when their trigger phrases apply.”

Claude.ai, plans with Skills

Settings → Capabilities → Skills. Upload the skill folder as a ZIP. Toggle it on. It loads itself when the description matches.

Claude Code and Cowork

Put the folder at .claude/skills/<name>/SKILL.md in your project. It appears as /<name>. Nothing to upload.

ChatGPT

Any chat, any plan

Paste SKILL.md after the one-line instruction on the next slide. Lasts one conversation.

A Project

Create a Project. Upload the SKILL.md files. In the instructions: “Follow the attached skills when their trigger phrases apply. Say which one you are using.”

A custom GPT

Create a GPT. Upload the skills as Knowledge. Same line in its instructions. Share it with your team.

Pick one row. Load one skill. Check it.

Open source, use them after class: github.com/mrhinkle/aienterprise-skills · github.com/mrhinkle/VibeCoding-a-Website

#	FEATURE YOU GET	SKILL
1	Your own colors, fonts, and look, recorded so they never drift	`site-customizer`
2	A headline and About section that sell without sounding salesy	`copy-editor`
3	Writing that sounds like a person, not a chatbot	`ai-slop-killer`
4	A prompt for a hero or background image	`brand-to-images`
5	A contact form that actually delivers	`aie-web-forms`
6	Your name finds your site in Google and in AI answers	`aie-web-seo`
7	Every link, image, and anchor works	`aie-web-links`
8	A usability review from a stranger's point of view	`aie-web-ux`
9	The "does it actually work" gate before you share it	`aie-web-qa`
10	A LinkedIn post that announces the site in your voice	`aie-social-linkedin`

PASTE THE SKILL BELOW THIS

```
Use the following
instructions as a skill for
my next request. Read them,
confirm what you will and
will not do, then wait for
my task.
```

Raw links and task prompts: docs/website-skills-menu.md. Save a copy of index.html first. One skill, one task, one complete file.

LUNCH

12:00 – 12:45

Back at 12:45. The afternoon is the game.

WHEN YOU SIT BACK DOWN, HAVE THIS

1. Your website URL in project-state.md
2. examples/alien-tic-tac-toe/checkpoint-3-complete/index.html open; play one round
3. Think about two emoji that are easy to tell apart

Two tracks after lunch. Same gates, same deadline.

Both tracks: a brief with checks by 1:40, a pull request per increment, live by 4:35.

TRACK A · THE GUIDED GAME

Themed tic-tac-toe battle game

Tested checkpoints at every stage. Eight themes.

Board, then battle, then connected. Theme, playtest, publish.

Pick it if you want to be sure you ship.

TRACK B · YOUR OWN SMALL PROJECT

The thing you brought

A tool for your team, a dashboard, a prototype.

One screen, one input, one result. Nothing more today.

Built with v0, one pull request per increment. Back tables.

Track B: one screen, one input, one result.

Write the “later” list in project-state.md. Half of it you will not need.

FITS VERSION ONE

A meeting-cost calculator.

A which-form-do-I-need chooser for your team.

A checklist, or a dashboard you paste numbers into.

A pitch page with a signup button that is a mailto link.

WAITS FOR LATER, NO MATTER HOW REASONABLE

Sign-in.

A database, payments, or email sending.

Anything that calls another company’s system.

Anything that stores other people’s data.

Track B: plan mode again, from scratch.

Biggest model. Fill the scope template, draw one wireframe, type this. Cut anything not on the wireframe.

12:50 · BRIEF AND PLAN · ATTACH SCOPE + WIREFRAME · VERBATIM

```
Here is my brief, scope, and a wireframe. Version 1 is one screen, one input, one result, no
sign-in, no database, no external services. Tell me the plan: which files you will create, what
each does, and three checks I can perform by hand to know it works. Do not build yet.
```

Track B: build increment 1, then the afternoon.

1:40 · BUILD INCREMENT 1 · VERBATIM

```
Build the plan as described. Keep it to the files you listed. Plain, readable code with short comments. Fits a phone. Return the complete files.
```

1:40	Increment 1. Deploy. Three checks pass on the live URL.
3:00	One improvement through a pull request. A two-minute stranger test. One fix through another PR.
4:10	Publish. Five-line README. The “later” list in project-state.md.
3:30, nothing live?	Switch to game checkpoint 3 and ship that. Come back to your idea after class.

Pick one of three games.

The rest of this deck follows the featured game. No accounts, databases, leaderboards, or downloaded art in class.

FEATURED

Alien Tic-Tac-Toe

The player chooses a board cell and wins or loses a short alien battle to claim it.

Why: a familiar board plus a small battle system.

Test: a battle awards exactly one cell and the board ends correctly.

SIMPLEST

Five-question quiz

The player answers one question at a time, sees feedback, and tries again.

Why: the simplest complete game, no movement. A good scope cut if you are behind.

Test: one answer scores once; final score and replay are right.

MOST MOTION

One-button space dodger

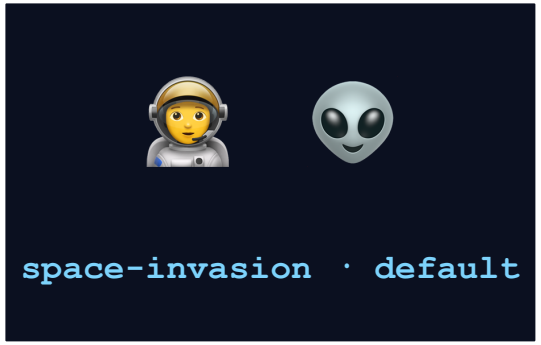
The player taps or presses a key to rise between obstacles.

Why: one control and a theme you can customize. The most motion to debug.

Test: motion, collisions, scoring, and restart work every time.

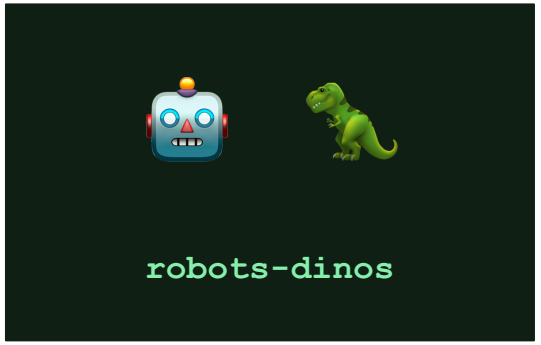
Same game, eight looks. Renaming is all a theme does.

Try one on the checkpoint: checkpoint-3-complete/index.html?theme=ghosts-pumpkins

















Making your own? Fill this in first.

FIELD	YOUR ANSWER
Theme id (lowercase, hyphens)	
You (emoji + name)	
Opponent (emoji + name)	
Attack 1 → counter 1	
Attack 2 → counter 2	
Attack 3 → counter 3	
Background, accent, and text colors	

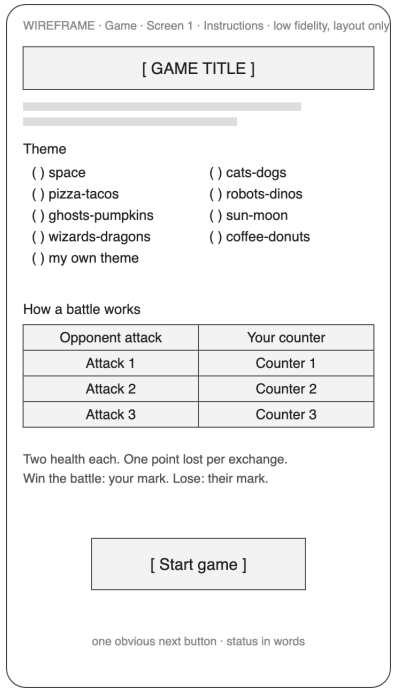
AVOID

Pairs that look alike (🐱 and 🐯).

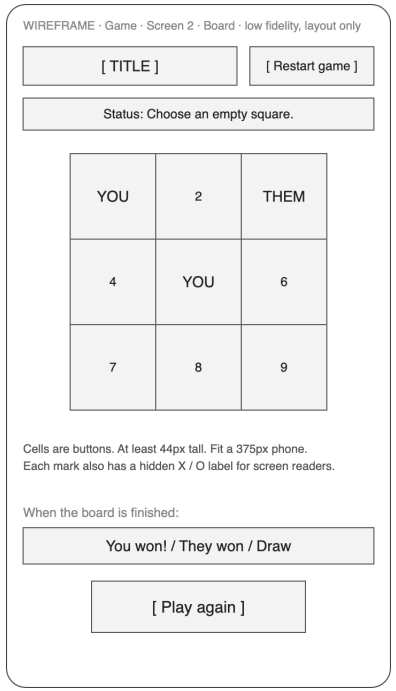
One emoji that is mostly the background color.

Anything that only reads as “mine” versus “theirs” by color.

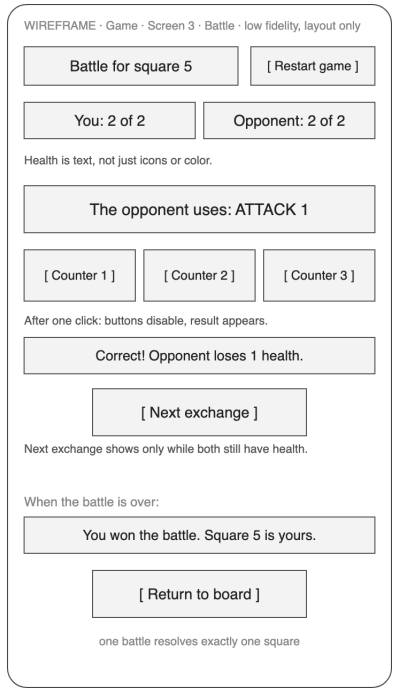
Draw the three screens first. Five minutes.



1 · Instructions



2 · Board



3 · Battle

EVERY SKETCH HAS

A title, a status line in words, one obvious next button.

Health written as text: “You: 2 of 2.”

Nothing that depends on color alone.

A board that fits a phone held upright.

Describe the game once, with your theme.

Done when: game-brief.md has your theme and at least five checks you could show a partner. Nothing built yet.

ATTACH alien-tic-tac-toe.md + vc-game-brief · REPLACE THE BRACKETS · VERBATIM

```
Write game-brief.md for a themed tic-tac-toe battle game using the rules in alien-tic-tac-toe.md exactly. Theme: I am [🐱 Cat], the opponent is [🐶 Dog]. The opponent's attacks are [Bark, Pounce, Slobber] and my correct counters are [Hiss, Climb, Shake Off], in that order. Colors: [dark brown background, peach accent, light text]. Rename only; do not change the health, exchange, or board rules. Give me observable completion checks. Do not build yet.
```

Observable means you can see it on screen: “three of my marks in a row shows You won.” If a check isn’t observable, ask for one that is.

Green sticky on your lid = done

Red sticky = a helper comes to you

List every state the game can be in.

This list is what you test against all afternoon.

PASTE YOUR THREE WIREFRAMES BELOW IT · TYPE THIS, VERBATIM

```
Using the brief, list every state the game can be in and what the player sees and can do in each. Use my three wireframes below as the screens. Confirm there is no automatic opponent move after a battle and that Restart game is available in every active state.
```

TWO THINGS IT MUST CONFIRM

1. No automatic opponent move after a battle.
2. Restart game is available in every active state.

Build the board alone.

Behind at 2:00? Download checkpoint-1-board/index.html and build the battle next.

PASTE THE BOARD WIREFRAME BELOW IT · REPLACE THE BRACKETS · VERBATIM

```
Build only the board screen as one index.html with inline CSS and JavaScript and no external
files. Cells show [🐱] for my marks and [🐶] for the opponent's, each with a visually hidden "X"
or "O" label. For now, two buttons "Award to me" and "Award to opponent" stand in for the battle.
Detect the eight lines, a draw, Play again, and Restart game. Return the complete file.
```

CHECKPOINT 1 · WHAT YOU MUST SEE

1 Three of your marks in a row shows “You won.” Three of theirs: you lost.

2 A full board with no line shows Draw. Play again and Restart both work.

3 Test the diagonals: 1-5-9 and 3-5-7.

Green sticky on your lid = done

Red sticky = a helper comes to you

Build the battle as its own file, on a branch.

This one goes through a pull request: branch, upload, “Compare & pull request,” read message, files, lines, then merge.

PASTE THE BATTLE WIREFRAME BELOW · REPLACE THE BRACKETS · VERBATIM

```
Build the battle screen as its own index.html. Both sides start with 2 health shown as text. The opponent's attacks cycle [Bark → Pounce → Slobber]. One accepted answer per exchange; disable the answer buttons after I click; show an explicit Next exchange button only while both sides still have health; end immediately when either side reaches zero. Return the complete file.
```

CHECKPOINT 2 · WHAT YOU MUST SEE

1. Answer correctly twice: the battle ends in two exchanges, with no Next exchange button after the win.
2. A wrong answer costs you one health, shown as text.
3. Buttons disable after one click.

Green sticky on your lid = done

Red sticky = a helper comes to you

Stuck? Grab the rung for where you are.

Say what you see. Try the fix in stuck.md. Grab a checkpoint. Raise a hand. Using one is not cheating.

WHERE YOU ARE	DOWNLOAD THIS	THEN CONTINUE WITH
Website: nothing works yet	`personal-website/index.html`	Replace Jordan Rivera's content with yours, section by section.
Website: content in, look is broken	`the same file, fresh`	Paste your text into the fresh file instead of fixing the broken one.
Website: looks fine, cannot publish	`your own file`	Setup guide: recover from common setup failures.
Game: board does not detect wins	`checkpoint-1-board`	Build the battle next.
Game: battle logic is wrong	`checkpoint-2-battle`	Connect it to the board.
Game: board and battle will not connect	`checkpoint-3-complete`	Pick your theme with ?theme= and go straight to look and feel and playtest.
Game: everything works but it is 4:30	`checkpoint-3-complete`	Change the theme, publish it, record the rest as "later."

Connect them. Now it’s a game.

Compare with checkpoint-3-complete/index.html. At 3:00 everyone may continue from it.

ATTACH BOTH FILES + THE INSTRUCTIONS WIREFRAME · TYPE THIS, VERBATIM

```
Connect the board and the battle into one index.html. Selecting an empty cell locks the board and starts a battle for that cell. A battle win places my mark, a loss places the opponent's mark, exactly once, then the board is evaluated. Add the instructions screen from my wireframe with the theme table. Keep Restart game available everywhere. Return the complete file.
```

CHECKPOINT 3 · WHAT YOU MUST SEE

1. Win three battles on squares 1, 2, and 3: the game says you won.
2. Restart in the middle of a battle: empty board, no battle.
3. One finished battle fills exactly one cell.

Green sticky on your lid = done

Red sticky = a helper comes to you

BREAK

2:45 – 3:00

Back at 3:00. Then theme, phone fit, and playtest.

WHEN YOU SIT BACK DOWN, HAVE THIS

1. A playable round, or checkpoint 3 open with your ?theme=
2. Your three colors written down
3. Swap who types

Your colors, and it fits a phone.

Then play one round on a phone, held upright, with your thumb.

ATTACH YOUR CURRENT index.html · TYPE THIS, VERBATIM

```
Apply my theme colors and make the board fit a 375 pixel wide phone. Buttons at least
44 pixels tall. Keep every status message in words. Do not change any rule.
```

THEN FEEDBACK: EVERY CLICK ANSWERS YOU

Every click answers	“Correct! Dog −1.” “Square 5 is yours.” “You won!” In words, not only color.
Health is text	“You: 2 of 2.” Icons may reinforce it; they never replace it.
One obvious next button	Next exchange, Return to board, Play again. Never two choices at once.

Green sticky on your lid = done

Red sticky = a helper comes to you

Swap laptops. Say nothing. Watch. Fix one thing.

If you have to explain the game to your tester, that is the finding.

1. Swap with the pair next to you. Hand over the phone or laptop. Say nothing.
2. Watch one full round. Write the moment they hesitate or click wrong.
3. Describe that moment in the prompt. Make the one change it proposes.
4. Re-test exactly as told. Then swap back and do theirs.

REPLACE THE BRACKET WITH WHAT YOU SAW · TYPE THIS, VERBATIM

```
Watch me describe a partner's playtest: [what confused them]. Propose the single smallest change that fixes it. Change one thing, then tell me exactly how to re-test it.
```

Green sticky on your lid = done

Red sticky = a helper comes to you

A second repository: my-first-game.

Same moves as this morning. Two projects, two repositories, two URLs. The game never goes in the website repository.

1. GitHub: “+” → “New repository.” Name: my-first-game. Add a README.
2. “Add file” → “Upload files.” index.html at the top level, lowercase.
3. Commit: “First playable version.”
4. Never upload the game into my-first-site. That replaces your website.

Green sticky on your lid = done

Red sticky = a helper comes to you

A second Vercel project, a second live URL.

Deliverable two: a second live *.vercel.app address that plays a round on a phone.

1 “Add New” → “Project” → Import my-first-game.

2 Framework: Other. Build Command: override on, empty. Output: root.

3 Deploy. Visit. Play one round on a phone.

4 Try ?theme=your-theme on the live address. Record the URL in project-state.md.

Green sticky on your lid = done

Red sticky = a helper comes to you

One skill for your site, or one game extension.

Skills menu: ten rows, same rules as sprint 1. Suggested: #5, the contact form, or #6, SEO. Or one extension below.

GAME EXTENSION	WHAT CHANGES	CHECK
Random attacks	The opponent picks any attack each exchange instead of cycling.	Over ten exchanges you see all three attacks.
Score across boards	Count boards won and lost in the session.	Play again keeps the tally; Restart during a board does not count it.
Two themes at once	The player chooses the opponent separately from themselves.	The counter table updates to match.
Sound	One short sound on win and one on loss, with a mute button.	Muted stays muted after Play again.

Not in class: accounts, leaderboards, a database, timers, downloaded art. Show and tell at 5:05.

Green sticky on your lid = done

Red sticky = a helper comes to you

Sixty seconds. Your live URL. One thing that broke.

Everyone else: one thing you noticed, on a sticky, handed to them.

1

Your live URL on the big screen

Not the file on your laptop. The URL went through your gate.

2

One move or one scroll

Play one battle, or scroll to your About section.

3

One thing that broke, and what fixed it

The most useful sentence of the day. Everyone got stuck there too.

Tuesday morning. Three decisions to make before lunch.

The pull request is how you supervise. Smallest repeatable task first. Which tool after today: docs/platform-landscape.md.

1

What goes in GitHub first

The thing your team edits most and backs up least. A checklist, a process doc. Put it in a repository.

2

What to automate next

One repeatable task, small enough to check by eye. The report you rebuild every Monday.

3

Where agents still need adult supervision

Secrets and keys. Customer data. Anything customer-facing. Anything you can’t test yourself.

Write it down so Monday-you can continue.

Unfinished is fine. Say what works and what doesn’t. Never relabel it as done.

PROJECT-STATE.MD · ONE PER PROJECT

Project, repository, branch, and version.

Vercel project and URL.

What you tested and what you saw.

Current blocker, and one next action with its check.

YOU LEAVE WITH

A live website and a live game, in your Vercel.

Two repositories, in your GitHub.

Two state files that let any chat continue.

A loop you can run on Monday: describe, plan, build, check, save, publish.

Optional: post about today and I'll endorse the skill.

Recognition of a skill you practiced today, not a certification. You choose what to publish.

1. Add "Vibe Coding" in your profile's Skills section. Can't find it? Ask me which label to use.
2. Connect with me on LinkedIn, using the profile I show you now, not a guess.
3. Publish a post in your own words: something you built, learned, or worked through.
4. Send me your profile URL, post URL, and the exact skill name. I endorse it personally.

ASK YOUR CHATBOT FOR A DRAFT · VERBATIM

```
Help me draft a short LinkedIn post about this class using only these facts: [what I built, what works, what I learned, and a challenge]. Keep my voice. Mention that Mark offered a skill endorsement to participants who post about their experience. Do not invent outcomes, praise, or expertise. Give me a draft to review; do not publish it or message anyone.
```

Everyone gets the slides. Reviewers get the book.

EVERYONE GETS THE SLIDES

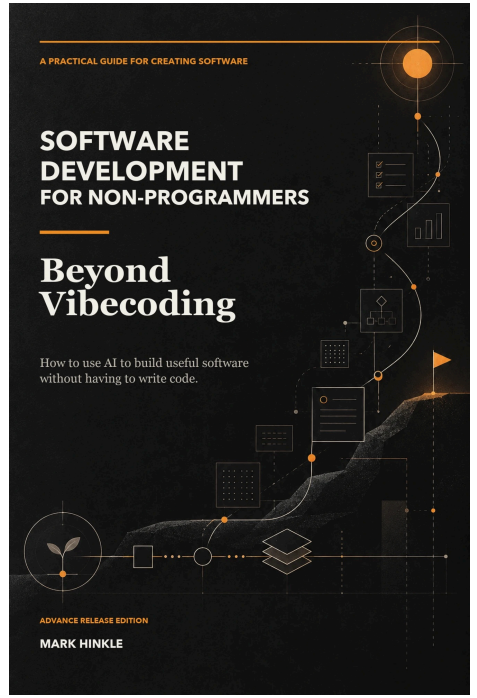
The slides and all the course materials are in the repo. PDF and PPTX in presentations/ato-2026-workshop.

github.com/mrhinkle/Vibe-Coding-for-Non-Technical-Professionals

REVIEWERS GET THE BOOK

Send me feedback, good or bad, and you get the preview release edition of Software Development for Non-Programmers: Beyond Vibecoding.

```
To: mrhinkle@allthingsai.org
Subject: ATO workshop feedback
Or hand me the printed sheet before you leave.
```



Advance Release Edition

See you in the spring: All Things AI.

April 18–20, 2027

All Things AI 2027, in this same building: the Atlantic Union Bank Convention Center.

April 18 is the Agent Hackathon. April 21 is a separate, screened Executive Summit.

allthingsai.org



THAT'S THE DAY

Now it's two URLs you own.

A site about you, live in your Vercel. A game you drew on paper at one o'clock, playing on a phone.

Run the same six steps on Monday, on something at work.

MARK HINKLE

mrhinkle@allthingsai.org

All Things AI · allthingsai.org The AIE Network · theaienterprise.io

Feedback sheets on the table on your way out. Subject line: ATO workshop feedback.

Slides and all course materials: github.com/mrhinkle/Vibe-Coding-for-Non-Technical-Professionals